

Learning Whittle and Gittins Indices using Reinforcement Learning

Problem setting: RMAB (infinite horizon, discounted)

- N arms, each with state s_i
- At each step, select $L < N$ arms (activate)
- Remaining arms are passive
- **Restless:** passive arms still transition

Objective (infinite-horizon discounted MDP): with fixed $\gamma \in (0, 1)$,

$$\max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right].$$

There is **no finite planning horizon** T : the sum is the usual discounted infinite-horizon return. The simulator step budget is only **RL training length**, not a truncated-horizon reward.

- π : policy (which arms to select)
- γ : discount factor (same γ in Bellman backups, VI truth, and Gittins map $(1 - \gamma)M$)

Challenge: Joint optimization over all arms is PSPACE-hard

- Avoid the full joint MDP: solve a *per-arm* surrogate
- Assign an **index** to each arm state
- Activate the L arms with largest indices

Types:

- Gittins index (rested bandits)
- Whittle index (restless bandits)

Key idea: Reduce decision-making to a ranking problem

Whittle index: intuition

- Same **infinite-horizon discounted** setting: all continuation values use γ in Q -/VI backups
- Introduce a subsidy λ for the passive action
- Passive flow reward includes λ ; active does not
- Compare $Q(s, 1)$ (activate) vs. $Q(s, 0)$ (passive) in the subsidized problem

Whittle condition (index state):

$$Q(s, 1) = Q(s, 0)$$

- $\lambda^*(s)$ satisfying the equality is the Whittle index at s

Interpretation:

- Larger $\lambda \Rightarrow$ passive relatively more attractive
- Smaller $\lambda \Rightarrow$ prefer to activate

QWL: augmented Q-learning for Whittle

- Augmented $Q^*(s, a)$ for each **reference state** x (subsidy tied to $\lambda(x)$) plus vector $\lambda(\cdot)$.
- **Two time scales:** $\alpha(n)$ fast on Q ; $\beta(n) = o(\alpha(n))$ slow on λ (e.g. $\alpha(n) = 1/\text{ceil } n/5000$, β sparse / smaller).

Optional **double Q-learning**: maintain Q^A, Q^B ; clip Q and λ if desired. Each environment step typically updates the pulled arm with $a=1$ and other arms passively with $a=0$, $r=0$.

QWI: Q- and λ -updates

Fast time scale (Q-learning), arm i , step n :

$$Q_{n+1}^x(s_n^i, a_n^i) = (1 - \alpha(n)) Q_n^x(s_n^i, a_n^i) + \alpha(n) \left[(1 - a_n^i)(r_0(s_n^i) + \lambda_n(x)) + a_n^i r_1(s_n^i) + \gamma \max_{v \in \{0,1\}} Q_n^x(s_{n+1}^i, v) \right].$$

Slow time scale (Whittle index), all references x :

$$\lambda_{n+1}(x) = \lambda_n(x) + \beta(n) (Q_n^x(x, 1) - Q_n^x(x, 0)).$$

Passive reward includes subsidy λ ; active uses r_1 . Often $r_0 \equiv 0$ and only the pulled arm carries immediate reward r , giving $(1-a)\lambda(x) + ar$ in the backup.

QWI: fixed point and double Q-learning

Whittle state: at reference x , index $\lambda^*(x)$ solves $Q^x(x, 1) = Q^x(x, 0)$.

Index step intuition: if $Q^x(x, 1) > Q^x(x, 0)$, increase $\lambda(x)$ until indifference.

Double Q-learning (homogeneous tabular): with probability $\frac{1}{2}$ update Q^A using $\arg \max_v Q^B(x, s', v)$ in the target (swap $A \leftrightarrow B$ otherwise). The index gap can use the average of $(Q^A(x, x, 1) - Q^A(x, x, 0))$ and $(Q^B(x, x, 1) - Q^B(x, x, 0))$. Without double Q-learning, one may bootstrap with the mean of $Q(x, s', \cdot)$ over actions instead of max.

Algorithm: QWI (tabular Whittle)

Input: $\gamma \in (0, 1)$, ε , schedules $\alpha(n), \beta(n)$, horizon n_{end} .

Output: $\lambda(x)$ for all $x \in \mathcal{S}$ (per arm if heterogeneous).

- 1 Initialize s_0 for all arms; $Q^x(s, a) \leftarrow 0$, $\lambda(x) \leftarrow 0$.
- 2 for $n = 1$ to n_{end} :
 - 1 **Action:** ε -greedy — activate L arms with largest $\lambda_n(s^{(k)})$ (or random).
 - 2 **Observe** rewards and next states for all arms.
 - 3 Set learning rates $\alpha(n)$, $\beta(n)$ (e.g. β only on a sparse subsequence, $\beta=o(\alpha)$).
 - 4 **Q-update:** for each arm i , visited (s_n^i, a_n^i) , **for all** references x , TD toward

$$(1 - a_n^i)(r_0 + \lambda_n(x)) + a_n^i r_1 + \gamma \max_v Q_n^x(s_{n+1}^i, v)$$

(optionally replace max with double-Q or clipped targets).

- 5 **λ -update:** if $\beta(n) > 0$, for all x : $\lambda_{n+1}(x) = \lambda_n(x) + \beta(n)(Q_n^x(x, 1) - Q_n^x(x, 0))$ (clip if used).
- 3 return λ .

Worst-case work per step scales as $\mathcal{O}(|\mathcal{S}|^2)$ when every reference x is updated each transition.

QWINN: neural Whittle learner

- $Q_\theta(s, x, a)$: inputs are **visited state** s and **reference** x (often encoded as features or one-hots); outputs $(Q(\cdot, 0), Q(\cdot, 1))$.
- **Target network** $Q_{\theta'}$ for the bootstrap max; Polyak averaging $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$ after optimizer steps.
- Same **slow** λ as QWI; replay stores active *and* passive transitions.

QWINN: targets, loss, index

TD target (frozen θ'): for sample (s, a, r, s') at reference x ,

$$Q_{\text{target}}^x(s, a) = (1-a)(r_0(s) + \lambda(x)) + a r_1(s) + \gamma \max_{v \in \{0,1\}} Q_{\theta'}^x(s', v).$$

Index update:

$$\lambda_{n+1}(x) = \lambda_n(x) + \beta(n) (Q_{\theta}^x(x, 1) - Q_{\theta}^x(x, 0)).$$

Training loss: e.g. Huber or MSE on the *active* head if $a=1$, passive head if $a=0$;

semi-gradient: no gradient through θ' inside the max term.

Typical implementation: aggregate loss over references x , detach the unused head per sample, update λ from online $Q_{\theta}(x, x, \cdot)$.

Algorithm: QWINN (neural Whittle)

Input: γ, ε , buffer, batch B , update period κ , Polyak τ , Adam on θ .

Output: $\lambda(x)$.

- 1 Init θ, θ' (same), $\lambda \leftarrow 0$, replay $\mathcal{D}$.
- 2 for $n = 1, 2, \dots$:
 - 1 Select arms (ε -greedy on $\lambda(s^{(k)})$); env step.
 - 2 Push $(s_{\text{pull}}, 1, r, s'_{\text{pull}})$; for each idle arm push $(s^{(i)}, 0, 0, s'^{(i)})$.
 - 3 if $n \bmod \kappa = 0$ and $|\mathcal{D}| \geq B$: sample minibatch; for each $x \in \mathcal{S}$ build targets with $Q_{\theta'}$ and current $\lambda(x)$ as above; accumulate loss; Adam step; Polyak update θ' .
 - 4 if $\beta(n) > 0$: $\lambda(x) \leftarrow \text{clip}(\lambda(x) + \beta(n)(Q_{\theta}(x, x, 1) - Q_{\theta}(x, x, 0)))$ for all x .

QWINN: Bellman error $E(\theta, \tau)$

The DQN-style objective can be written as minimizing

$$E(\theta, \tau) = \mathbb{E} \left[\left(Q_{\theta}^x(s, a) - (1-a)(r_0(s) + \lambda) - a r_1(s) - \gamma \max_v Q_{\tau}^x(s', v) \right)^2 \right],$$

where θ are online weights and τ are **frozen** target weights (no gradient through τ in the max term — semi-gradient). Local convergence analyses use the Hessian of E in θ and contraction-style conditions on how τ tracks θ .

Gittins index: intuition (rested bandit)

- Reformulate as an optimal *stopping* problem

Actions:

- Continue (pull the arm)
- Retire and collect a lump payoff W (reservation value)

$$V(x, W) = \max(Q(x, 1), W)$$

Index as threshold $M(x)$:

$$V(x, M(x)) = M(x)$$

Link: $G(x) = (1 - \gamma)M(x)$ is the usual Gittins index (same map as in the QGI slides)

- **Retirement / stopping form:** continuation value under “always pull” vs. retire with lump $M(x)$.
- Learn **active** continuation $Q^x(s)$ (pull-only) for each reference x ; scalar retirement level $M(x)$.
- Arm ranking uses $M(s)$ at the observed state (same as Gittins threshold logic).

QGI: Bellman and index updates

After pulling an arm, observe (s, r, s') . For **every** reference $x \in \{0, \dots, N-1\}$ (homogeneous tabular entry $Q^x(s)$):

$$T^x = r + \gamma \max\{Q^x(s'), M(x)\}, \quad Q^x(s) \leftarrow (1 - \alpha(n)) Q^x(s) + \alpha(n) T^x.$$

When $\beta(n) > 0$, threshold move (slow time scale):

$$M(x) \leftarrow M(x) + \beta(n) (Q^x(x) - M(x)).$$

Reported Gittins index: $G(x) = (1 - \gamma) M(x)$.

Heterogeneous arms: use $Q_k^x(s)$, $M_k(x)$ and the pulled arm index k .

Algorithm: QGI (tabular Gittins)

Input: $\gamma, \alpha(n), \beta(n), \varepsilon, n_{\text{steps}}$.

Output: $G(x) = (1 - \gamma)M(x)$.

- ➊ Init $Q^x(s) \leftarrow 0, M(x) \leftarrow 0$ (per arm if heterogeneous).
- ➋ for each RL step:
 - ➊ ε -greedy arm: greedy on $M(s^{(k)})$.
 - ➋ Env step $\Rightarrow$ pulled arm data (s, r, s') only (idle arms not replayed).
 - ➌ for all x : TD target $r + \gamma \max\{Q^x(s'), M(x)\}$; update $Q^x(s)$.
 - ➍ if $\beta(n) > 0$: $M(x) \leftarrow M(x) + \beta(n)(Q^x(x) - M(x))$ for all x .
- ➌ return $G(x) = (1 - \gamma)M(x)$.

DGN: deep Gittins learner

- Scalar network $\hat{Q}_\theta(s, x)$ approximates the **active** continuation value (“pull” value) for reference x .
- Targets use frozen $\hat{Q}_{\theta'}$ and **current** retirement levels $M(x)$, identical structure to QGI tabular TD.
- $M(x)$ updated slowly toward $\hat{Q}_\theta(x, x)$; export $G(x) = (1 - \gamma)M(x)$.

DGN: targets and threshold

For each reference x , one-step target on sample (s, r, s') :

$$Y^x = r + \gamma \max\{\hat{Q}_{\theta'}(s', x), M(x)\}, \quad \mathcal{L}^x = \sum_{(s, r, s') \in \mathcal{B}} \rho(\hat{Q}_{\theta}(s, x), Y^x)$$

with ρ a Huber- or L_2 -type loss. **Polyak:** $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$ after each train step.

When $\beta(n) > 0$:

$$M(x) \leftarrow M(x) + \beta(n) (\hat{Q}_{\theta}(x, x) - M(x)), \quad G(x) = (1 - \gamma)M(x).$$

Buffer stores **active-only** transitions from the pulled arm; ranking uses $M(s)$ like QGI.

Algorithm: DGN (neural Gittins)

Input: γ, ε , buffer, batch B , period κ , Polyak τ , Adam.

Output: $G(x) = (1 - \gamma)M(x)$.

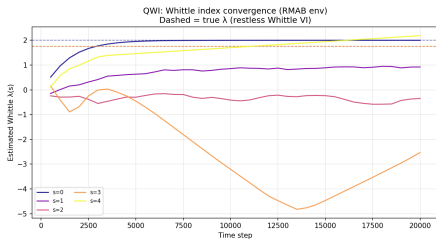
- 1 Init $\theta, \theta', M \leftarrow 0$, replay $\mathcal{D}$.
- 2 for each RL step n :
 - 1 ε -greedy arm on $M(s^{(k)})$; env step; append pulled arm tuple (s, r, s') to $\mathcal{D}$.
 - 2 if $n \bmod \kappa = 0$ and $|\mathcal{D}| \geq B$: sample batch $\mathcal{B}$; for each x build vector targets $r_j + \gamma \max(\hat{Q}_{\theta'}(s'_j, x), M(x))$; sum SmoothL1 vs. $\hat{Q}_{\theta}(s_j, x)$; Adam; Polyak θ' .
 - 3 if $\beta(n) > 0$: $M(x) \leftarrow M(x) + \beta(n)(\hat{Q}_{\theta}(x, x) - M(x))$ for all x .
- 3 return $G(x) = (1 - \gamma)M(x)$.

Evaluation metrics

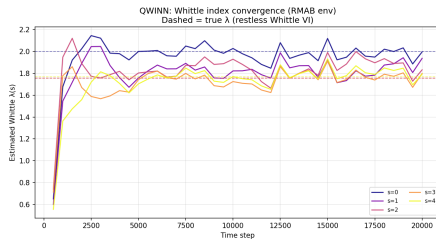
- **Truth indices:** λ^* (Whittle) and G^* (Gittins) from VI use the **same infinite-horizon discounted** criterion and γ as the learners.
- **Index error (MAE):** e.g. $\frac{1}{|S|} \sum_s |\hat{\lambda}(s) - \lambda^*(s)|$ or $|\hat{G}(s) - G^*(s)|$, averaged over s
- **Index MSE:** mean $\frac{1}{|S|} \sum_s (\hat{\lambda}(s) - \lambda^*(s))^2$ or the same with $(\hat{G}, G^*)$
- **Bellman relative error (BRE):** mean $|Q - \mathcal{T}Q|/(|\mathcal{T}Q| + \varepsilon)$ over visited (s, a)
- **Runtime:** wall-clock per algorithm; scaling vs. state size N when sweeps are run

Figure: Whittle index trajectories (QWI vs. QWINN)

QWI



QWINN



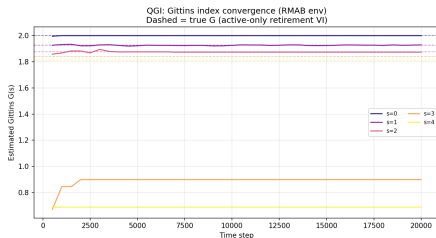
Solid = $\lambda_t(s)$ learned online; dashed = $\lambda^*(s)$ from VI. Both move λ using the gap $Q(s=x, x, 1) - Q(s=x, x, 0)$.

Why the Whittle plots differ (simple technical reasons)

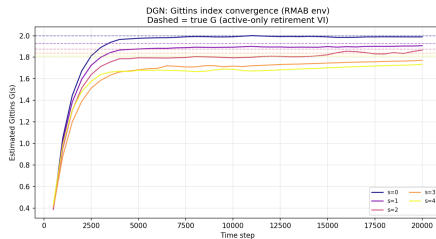
- **Same rule:** λ is pushed by how much the learned Q prefers *activate* vs. *passive* at state s . Wrong $Q \Rightarrow$ wrong λ .
- **QWI (table):** passive rewards in the backup use the current λ values. So bad Q and bad λ **feed each other** (errors grow on some states). If λ is **capped**, a flat line near a large value can mean “hit the cap,” not “found λ^* .”
- **QWINN (net):** one network represents Q for all states together, so estimates are **smoother** and less extreme. **Replay + target network** reduce TD/bootstrap noise, so the Q -gap stays small and λ **stays near** λ^* with jitter.
- **Wide axis on QWI:** a few states with huge λ stretch the y-scale; true λ^* can look like a tight bundle at the bottom even though they are correct.

Figure: Gittins index trajectories (QGI vs. DGN)

QGI



DGN

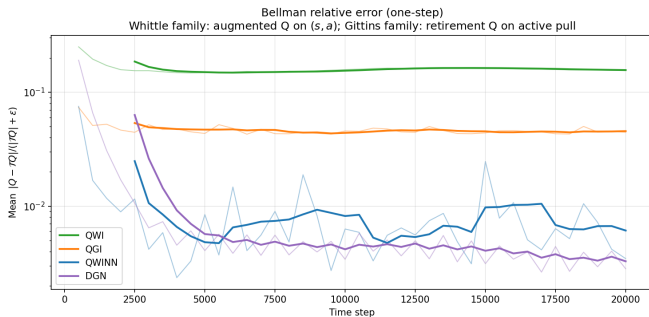


Solid = learned Gittins G ; dashed = VI truth G^* . Here learning uses **pull vs. retire** logic and a scalar threshold M per state, not the restless Whittle subsidy loop.

Why the Gittins plots look like this (simple technical reasons)

- **Simpler loop than Whittle:** only **active** Q is learned; “stop” is summarized by one number $M(s)$ per state. The subsidy does **not** enter every backup the way λ does in QWL, so the system is **less tangled** and usually easier to stabilize.
- **What moves the curve:** $M(s)$ is nudged toward the continuation value $Q(s, 1)$; then $G = (1 - \gamma)M$. When Q is near correct, G **tracks** the dashed G^* .
- **QGI wiggles:** each state is updated from **random** transitions (noise in Q), so you see small ups and downs even when learning is basically right.
- **DGN smoother:** the network averages information across states; minibatches and a **target network** cut noise, so lines are often **less jagged** than QGI.
- **Steady wrong level:** if exploration rarely visits “retire” vs. “pull,” or step sizes are off, $Q(s, 1)$ stays biased $\Rightarrow$ the whole G curve sits above or below the dashed truth.

Figure: Bellman relative error (all four)

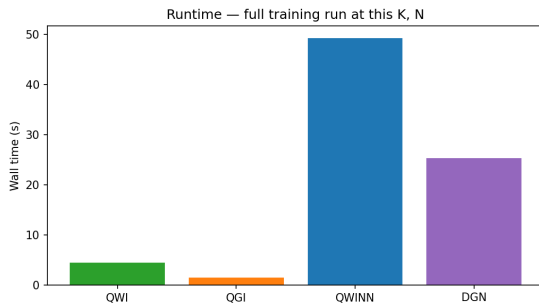


Each line = how far current Q is from a **one-step Bellman fixpoint** (relative error), log scale. Whittle learners and Gittins learners use **different** Bellman definitions, so line heights are not apples-to-apples.

Why the BRE curves look like this (simple technical reasons)

- **What drops fast at the start:** Q starts near zero; targets use rewards and bootstrapped Q' right away, so the mismatch is huge. After data accumulates, Q moves closer to its own backup $\Rightarrow$ the ratio **falls quickly** (this is normal, not magic).
- **Later flat region high:** Q still does not match its backup rule well—often because the **policy keeps changing** (exploration decay, arm choice) so the data do not match a single fixed Bellman problem yet.
- **Later flat region low:** backups are mostly consistent; **note:** for Whittle, small BRE still does not guarantee perfect λ , because λ reads a very specific part of Q (the diagonal gap).
- **Bumps:** exploration steps change which states are seen; target-network updates (especially in deep methods) can cause a **small jump** in the average residual.
- **Do not rank curves by height across colors:** Whittle and Gittins residuals are built from **different** Q -shapes and formulas.

Figure: Runtime (all four algorithms)



Each bar = seconds for one full run (same K , N , and training steps for all algorithms).

Why runtime looks like this (simple technical reasons)

- **QWI / QGI (tables):** many **nested loops** over states (and for QWI, over every “reference” state each step). Work grows quickly as N grows because the table gets bigger and each step touches more entries.
- **QWINN / DGN (nets):** each step does **matrix multiply + backprop** on a batch; that has a large **fixed cost** even for small N , so bars can be taller than tables at small problems.
- **Extra deep-RL work:** replay buffer sampling, target-network updates, and optimizer steps add time on top of the environment step.
- **Scaling curves vs. N :** table methods often **steepen** as N increases; nets widen the input but keep roughly the **same** weight matrix sizes, so lines can cross at larger N .
- **Important:** shorter time $\neq$ better index—it only means less compute was used; check the convergence plots for quality.

Key observations

- QWI: accurate in tabular settings; can be sample-heavy
- QWINN: generalizes; trades off index error vs. compute
- QGI: learns only active Q plus threshold $M(x)$; stable on rested benchmarks
- DGN: same Gittins targets with a neural approximator

Conclusion

- RMABs are hard to solve optimally at scale
- Index policies decouple ranking from the joint MDP
- RL can recover indices *without* an explicit model

Tradeoff:

- Tabular methods: strong guarantees in small $|S|$
- Deep methods: scalability and generalization